Usra – Technical Design

Nov 2, 2026 · @Aya Nasser

Build Usra in Flutter, offline-first, with Supabase (Postgres, Auth, Edge Functions) as the backend. Health records stay on the phone in v1. The code is split into feature modules behind small interfaces, so new record types, reminders, rules and AI features plug in without touching the core. The AI Doctor is a paid, opt-in add-on (credits + subscription). It runs through a Supabase Edge Function, so model keys never ship in the app.

Key decisions

1. **Flutter** over React Native. It gives pixel-identical Arabic/RTL layouts, smoother performance on budget Android, and mature on-device ML Kit and encrypted SQLite packages (comparison below).
2. **Supabase** for auth, entitlements, credits, push tokens and, later, sync. **Edge Functions** (TypeScript on Deno) hold all server logic: OCR fallback, AI Doctor, purchase webhooks, clinic APIs.
3. **Health data never sits readable in the cloud in v1.** Supabase Cloud has no Middle East region. Later sync is end-to-end encrypted, or uses **self-hosted Supabase** in an in-country region, to meet Egypt/KSA/UAE data rules.
4. **OCR on the phone first** (ML Kit + barcode + drug-list matching). A cloud fallback Edge Function handles handwriting and **Arabic, which ML Kit cannot read** (ML Kit languages).
5. **AI Doctor = rules first, LLM second.** Deterministic checks run on the phone. The LLM explains and spots patterns from a minimal, de-identified snapshot the user previews before sending.

Full stack

Layer	Choice	Why
Mobile framework	Flutter (Dart), one codebase for Android + iOS	RTL, performance, ML plugins (see comparison)
State + dependency injection	Riverpod	Testable, compile-safe, scales to many modules
Navigation	go_router	Deep links (clinic reminders, notifications)

Layer	Choice	Why
Models	freezed + json_serializable	Immutable data classes, generated JSON
Local database	Drift (SQLite) + SQLCipher	Typed queries, migrations, encrypted at rest
Secure keys	flutter_secure_storage (Android Keystore / iOS Keychain)	Family encryption key and recovery key
Files / photos	App sandbox, compressed WebP, encrypted per file	Photos are the largest data
On-device OCR	google_mlkit_text_recognition, google_mlkit_barcode_scanning; Apple Vision on iOS via a platform channel	Free, offline, private
Reminders	flutter_local_notifications + exact alarms (Android), local notifications (iOS), full-screen intent for alarms	Works offline, costs nothing
Home-screen widget	home_widget (+ native Kotlin/Swift widget UI)	"Today's medicines" widget
Charts	fl_chart	Spending, growth, lab trends
Localisation	flutter_localizations + intl (ARB files), Arabic first	RTL, Arabic-Indic digits, Hijri option
Backend	Supabase: Postgres, Auth, Storage, Realtime	One managed platform, open source (can self-host in-region later)
Server logic	**Supabase Edge Functions** (TypeScript, Deno)	OCR fallback, AI Doctor, purchase webhooks, clinic API
Auth	Supabase Auth: phone OTP via the Send SMS hook (any SMS provider, e.g. Unifonic), Apple / Google sign-in	Phone numbers are how people sign in in the region
Push	Firebase Cloud Messaging (APNs through FCM)	Caregiver alerts, clinic reminders
Payments	Google Play Billing + StoreKit via RevenueCat	AI credits and subscription; webhook into Supabase

Layer	Choice	Why
AI	LLM via Edge Function (e.g. Claude Sonnet 5.5 for analysis, Claude Haiku 5.5 for OCR clean-up)	Keys stay server-side; model swappable behind an interface
Cloud OCR fallback	Google Cloud Vision (document text) or a vision LLM, via Edge Function	Handwriting and Arabic
Crash reporting	Firebase Crashlytics	Free
Analytics	PostHog or Firebase Analytics, **no health data**	Product usage only
CI/CD	GitHub Actions + Codemagic (iOS builds without a Mac), Fastlane	Automated builds to both stores
Hot fixes	Shorebird (Flutter code push)	Ship urgent Dart fixes without a store review
Repo	GitHub `Opsedia-ai/Usra_medical_history`, mono-repo: `/app`, `/supabase`, `/docs`	One place for app, backend, docs

Flutter vs React Native

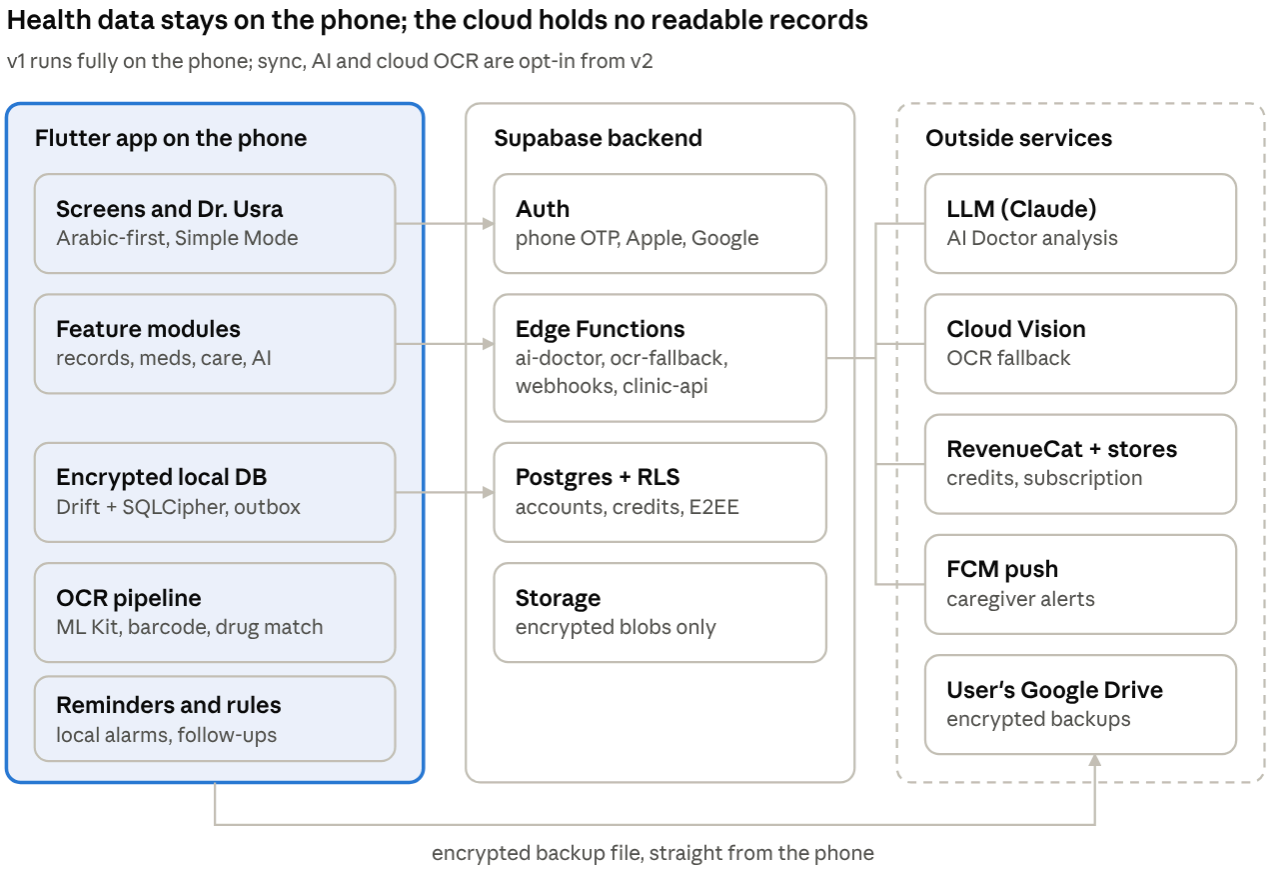
Recommendation: Flutter. Both can build Usra. Flutter wins on the things this app depends on most: identical Arabic/RTL screens on every phone, big-text elderly layouts that don't break, smooth performance on cheap Android, and ready plugins for on-device OCR and encrypted storage. React Native wins if the team is mainly JavaScript/TypeScript, or if over-the-air updates through Expo matter more.

Criterion	Flutter	React Native (with Expo)	Edge for Usra
Language	Dart	TypeScript	RN if the team knows TS; Dart is quick to learn
How UI is drawn	Own rendering engine: same pixels on every phone	Native platform widgets	**Flutter**: consistent elderly and RTL layouts across Android brands

Criterion	Flutter	React Native (with Expo)	Edge for Usra
Arabic / RTL	Built-in Directionality, mirrored widgets, Arabic-Indic digits	Supported via I18nManager; RTL switch usually needs an app restart	**Flutter**
Performance on budget Android	Compiled ahead of time to native code; steady frame rates	Good with the New Architecture; heavier JS work can lag	**Flutter** (slightly)
On-device OCR / barcode	Official-style ML Kit plugins (google_mlkit_*)	Community ML Kit libraries + Vision Camera frame processors	**Flutter**: simpler, more mature
Encrypted local DB	Drift + SQLCipher (typed, migrations)	op-sqlite / WatermelonDB with SQLCipher	Even; Drift is more pleasant
Exact alarms and full-screen alarms	flutter_local_notifications covers both	Notifee covers both	Even
Home-screen widgets	home_widget + native widget code	Native widget code + bridge	Even (both need native code)
Supabase SDK	Official supabase_flutter	Official supabase-js	Even
Over-the-air updates	Shorebird (paid tiers)	Expo EAS Update (mature)	**RN**
App size	Larger base app (bundled engine)	Smaller base	**RN** (matters on low-storage phones)
Web reuse (clinic portal)	Flutter Web works, but heavy for portals	Shares code and skills with a React web portal	**RN**; or build the portal separately in Next.js
Hiring in Egypt	Large Flutter community	Larger JS/React pool overall	RN slightly

What would change the call: if the founding developer is a strong React/TypeScript engineer, choose React Native + Expo. The architecture below works the same way in either; only the class syntax changes.

Architecture



architecture · phone, Supabase, outside services

The phone does all the work in v1. Supabase holds accounts, credits and (from v2) only encrypted blobs. Edge Functions are the single gateway to every outside service, so API keys never ship in the app.

Code structure and extension points

Every feature is a module that registers itself. The core knows nothing about vaccines, pregnancy or AI; it only knows modules, record types, schedules, rules and entitlements. Adding a feature = adding a folder and one line in the module list.

Folders (feature-first, clean architecture inside each feature)

```
app/lib/
  core/          di, database (Drift), crypto, sync, routing, l10n,
                 design_system (colours, severity, Dr. Usra), entitlements
  features/
    family/  records/  medications/  reminders/  followups/
    ocr/  spending/  insurance/  doctors/  child_care/
    pregnancy/  elder_mode/  emergency/  backup/  ai_doctor/
      each: data/ (DB + API)  domain/ (models, use cases)  presentation/
(screens)
  main.dart
supabase/
  migrations/    SQL + row-level security policies
  functions/     ocr-fallback, ai-doctor, revenuecat-webhook,
                 send-sms-hook, clinic-api
```

1. Feature modules

```
/// Every feature plugs in through this contract.
abstract class FeatureModule {
  String get id;
  List<RouteBase> get routes;
  List<RecordTypeDefinition> get recordTypes => const [];
  List<HomeCardBuilder> get homeCards => const [];
  List<FollowUpRule> get followUpRules => const [];
  Entitlement get requires => Entitlement.free;
  void register(ProviderContainer container) {}
}

final modules = <FeatureModule>[
  FamilyModule(), RecordsModule(), MedicationsModule(),
  RemindersModule(), ChildCareModule(), PregnancyModule(),
  ElderModeModule(), AiDoctorModule(),
];
```

2. One generic record, many types. Visits, scans, labs, vaccines and surgeries are all a `MedicalEvent`; each type adds its fields through a registry, so a new type needs no new table.

```
@freezed
class MedicalEvent with _$MedicalEvent {
  const factory MedicalEvent({
    required String id,            // UUID v7, sync-safe
    required String profileId,
    required String type,          // 'visit', 'surgery', 'vaccine'...
    required DateTime occurredAt,
    String? doctorId,
```

```
    Money? cost,
    String? insurancePolicyId,
    @Default({}) Map<String, Object?> data, // type-specific fields
    @Default([]) List<String> attachmentIds,
    required DateTime updatedAt,
    DateTime? deletedAt,           // soft delete for sync
  }) = _MedicalEvent;
}

class RecordTypeDefinition {
  const RecordTypeDefinition({required this.key, required this.icon,
      required this.fields, this.hasCost = true});
  final String key;
  final IconData icon;
  final List<FieldSpec> fields;    // drives the form and validation
  final bool hasCost;
}
```

3. Repositories hide storage. Screens never touch the database or Supabase directly, so moving from phone-only to cloud sync changes one class.

```
abstract interface class EventRepository {
  Stream<List<MedicalEvent>> watchTimeline(String profileId);
  Future<void> save(MedicalEvent event);   // local write + sync outbox
  Future<void> softDelete(String id);
}
```

4. Reminder schedules are strategies. "Every 4 hours", "daily at 8" and "2 weeks after surgery" are classes behind one interface.

```
abstract class Schedule {
  Iterable<DateTime> occurrences(DateTime from, DateTime to);
}
class EveryNHours extends Schedule { /* ... */ }
class DailyAt extends Schedule { /* ... */ }
class RelativeToEvent extends Schedule { /* ... */ }
```

5. OCR is a chain of steps. Each step either returns a confident result or passes on.

```
abstract class OcrStep {
  Future<OcrResult?> run(OcrInput input);   // null = try next step
}

class OcrPipeline {
  OcrPipeline(this.steps);  // [Barcode, OnDeviceText, DrugMatch,
CloudFallback]
  final List<OcrStep> steps;

  Future<OcrResult> process(OcrInput input) async {
    for (final step in steps) {
      final result = await step.run(input);
      if (result != null && result.isConfident) return result;
    }
    return OcrResult.manualEntry();
  }
}
```

6. Paid features check entitlements, not purchase code.

```
enum Entitlement { free, aiCredits, plus }

abstract interface class EntitlementService {
  Stream<Set<Entitlement>> watch();
  Future<int> aiCreditBalance();
}
```

Adding a feature, step by step: create `features/<name>/`, implement `FeatureModule`, register record types, routes, home cards and follow-up rules, add strings to the Arabic and English ARB files, write tests. The core does not change.

Data model and offline sync

The phone's encrypted database is the source of truth. Every table is built for sync from day one, so cloud sync can be switched on later without a migration of meaning.

Table	Holds	Where it lives
families, profiles, memberships	People, roles (owner, guardian, cared-for, viewer), private categories	Phone; membership metadata on Supabase when sync is on

Table	Holds	Where it lives
medical_events	Visits, scans, labs, vaccines, surgeries, stays (type + JSON fields)	Phone (encrypted)
attachments	Photo/PDF/video metadata; files stored separately	Phone (encrypted files)
medications, reminders, dose_log	Prescriptions, schedules, taken/snoozed/skipped history	Phone
followup_suggestions	Proposed plans from the rules engine; accepted/dismissed	Phone
doctors, facilities, insurance_policies	Directory, cards, coverage, validity	Phone
growth_measurements, pregnancies	Child growth points, pregnancy dates and checks	Phone
ai_reports	Saved AI Doctor findings	Phone
outbox	Changes waiting to sync	Phone only
accounts, entitlements, credits_ledger, push_tokens	Login, purchases, AI credit balance, devices	**Supabase** (no health data)

Sync rules (v2, when enabled)

- Every row has a UUID v7 id, `updated_at` and `deleted_at`. Writes go to the local DB and the outbox, then a background worker pushes them.
- **End-to-end encryption:** each record is encrypted on the phone with the family key (AES-256-GCM) before upload. Supabase stores only ciphertext, ids and timestamps, so it cannot read health data.
- The family key is shared by wrapping it with each member's device key when they accept an invite. Removing a member rotates the key.
- Conflicts: last writer wins per record, using server timestamps; dose logs are append-only, so they never conflict.
- Readable health data in the cloud (needed only for clinic partners in Phase 3) goes to a **self-hosted Supabase in an in-country region**, after legal review.

OCR pipeline

Most scans finish on the phone for free; only hard cases go to the cloud, with consent. The pipeline is the `OcrPipeline` chain above, with four steps:

1. **Barcode** (ML Kit barcode, on device): if the box has a barcode, look it up in the bundled drug table. Exact match, done.
2. **On-device text** (ML Kit Text Recognition on Android and iOS; Apple Vision as an extra on iOS): reads printed **Latin** text, which covers most brand names on boxes and printed prescriptions in Egypt. ML Kit does **not** read Arabic.
3. **Drug matching** (on device): normalise the text, then fuzzy-match against a bundled drug list per country (brand name, active ingredient, strength). For Egypt, built from the Egyptian Drug Authority registry. Suggestions show with a confidence score: "Did you mean *Augmentin 1 g*?".
4. **Cloud fallback** (`ocr-fallback` Edge Function), only if confidence is low, the text is Arabic, or it is handwriting, and only after the user taps "Improve with cloud":
   - The phone crops and compresses the image (~150 KB) and strips metadata.
   - The Edge Function calls a vision model (Google Cloud Vision document text, or a vision LLM such as Claude Haiku 5.5) and asks for **structured JSON**: name, strength, dose, frequency, duration.
   - The result goes back through drug matching on the phone; **the user always confirms** before anything is saved.
   - The image is not stored on the server; requests are rate-limited (e.g. 20 per member per month free).

Output of every path: a `MedicationDraft` the user edits and confirms, which then creates the medicine record and its reminders in one step.

Accuracy plan: collect (with opt-in consent) anonymous "was this suggestion right?" feedback to tune the drug-matching rules; no images leave the phone for this.

Reminders and follow-up rules engine

Reminders are data + a schedule strategy; follow-ups are rules that create reminders. Both run entirely on the phone.

- **Reminder** = what + who (profiles) + notify (people) + `Schedule` + delivery (normal, full-screen alarm, repeat until confirmed) + escalation (alert a guardian after N minutes).
- **Scheduler:** computes the next 64 occurrences and registers them as local notifications (iOS allows 64 pending). It recomputes after every change, at boot, and on timezone change.
- **Android reliability:** request exact-alarm permission, use full-screen intents for alarms, and walk the user through battery-saver settings per brand.

- **Dose log:** "Taken / Snooze / Skip" actions on the notification write to `dose_log`. Adherence reports and caregiver alerts come from it.
- **Caregiver escalation** (needs sync): when a dose is not confirmed in time, the phone (or a scheduled Edge Function, once sync is on) sends a push to the guardian.

Follow-up rules are small JSON files bundled with the app and updatable without a release:

```
{
  "id": "antibiotic-course-end",
  "trigger": { "on": "medication.created", "where": { "category":
"antibiotic" } },
  "suggest": [
    { "type": "reminder", "title": "Check the course is finished",
      "at": { "relativeTo": "medication.endDate", "offset": "P0D" } }
  ],
  "severity": "yellow",
  "source": "Usra care rules v1"
}
```

Rules cover cases (fever, surgery, abnormal lab, chronic condition) and ages (vaccines, well-child checks, adult and 65+ checks). A `RuleEngine` class matches events against rules and produces `FollowUpSuggestion`s; the user accepts, edits or dismisses each one.

AI Doctor assistant

"Dr. Usra AI" reviews the family's records and flags possible risks to discuss with a doctor; it never diagnoses or prescribes. It is opt-in and paid with credits. Each run sends only a minimal, de-identified summary that the user sees before sending.

How a run works

1. **Local checks first (free, on the phone):** duplicate active ingredients, overdue vaccines, missed doses, expiring medicines, out-of-range lab values against the lab's own range, growth flags. These are plain rules, not AI.
2. **Build the snapshot:** age (not birth date), sex, conditions, allergies, current medicines, last 12 months of labs and vitals as trends, adherence %, and the local check results. **No names, phone numbers, IDs, photos or free-text notes.**
3. **Preview and consent:** "This is what Dr. Usra AI will read. Cost: 1 credit." The user can untick any member or category.
4. **Edge Function `ai-doctor`:**
   - Verify the user's login.
   - Reserve credits atomically in Postgres.

- Call the LLM (e.g. Claude Sonnet 5.5) with a fixed safety prompt and the snapshot.
- Validate the reply against a JSON schema.
- Commit the credits, or refund them on failure.
- **Nothing from the snapshot is stored on the server**; only token counts and the credit transaction are logged.

5. **Show findings** as cards in the app's severity colours, each linked to the records it is based on, and saved locally in `ai_reports`.

```
{
  "findings": [
    {
      "member": "P2",
      "severity": "orange",
      "title": "Two paracetamol products at the same time",
      "why": "Panadol and Adol are both active this week and contain the same
ingredient.",
      "evidence": ["med_81", "med_92"],
      "action": "Ask your doctor or pharmacist which one to keep."
    }
  ],
  "urgent": false,
  "disclaimer": "Not a diagnosis. Talk to your doctor."
}
```

Safety rules

- If the user's question or the data suggests an emergency (chest pain, baby under 3 months with fever, pregnancy danger signs), the app shows the emergency screen immediately, without waiting for the AI.
- Findings use cautious wording ("possible", "worth asking your doctor") and always point to a human doctor; no doses, no new medicines.
- Record text is treated as data, never as instructions (guards against prompt injection from scanned notes).
- Every run is logged on the phone so families can see what was analysed and when.

Credits and subscription

Product	Type in the stores	What the user gets
Core app	Free	Everything except AI
AI credit pack	Consumable in-app purchase	e.g. 5 or 20 analyses, [price per market]

Product	Type in the stores	What the user gets
Usra Plus	Auto-renewing subscription	Monthly AI credits + encrypted cloud backup and family sync, [price per market]

- Purchases go through Google Play Billing / StoreKit (the stores require in-app purchase for digital features like this). RevenueCat handles receipts and sends a webhook to the `revenuecat-webhook` Edge Function, which writes to `credits_ledger`.
- The balance is computed in Postgres (a sum over the ledger) with row-level security; the app only reads it. One analysis costs a fixed number of credits regardless of family size, capped by a token budget per run.

Regulatory gate (must clear before launch):

- **Medical device rules:** a personalised risk analysis may count as software as a medical device in Saudi Arabia (SFDA guidance MDS-G27). Keep AI Doctor off in KSA until reviewed.
- **Egypt:** sending health data to a model hosted abroad is a cross-border transfer under the PDPL and needs a licence. Use de-identified snapshots and get the licence (or an in-region model endpoint) before launch.
- **Apple guideline 5.1.3:** health data may be used only to improve health management, with permission, and never for ads.

Security, privacy and compliance

- **At rest:** SQLCipher database and per-file encryption; keys in Android Keystore / iOS Keychain; app lock with biometrics.
- **In transit:** TLS everywhere; certificate pinning for Edge Function calls.
- **Cloud:** health data only as end-to-end-encrypted ciphertext in v2; Supabase row-level security on every table; service keys only inside Edge Functions; secrets in Supabase Vault.
- **Backups:** encrypted file to the user's own Google Drive (not iCloud: Apple guideline 5.1.3 forbids personal health data in iCloud).
- **Consent:** per-feature consent screens in Arabic and English (cloud OCR, AI Doctor, sync, clinic sharing), with guardians consenting for children.
- **Store requirements:** in-app account and data deletion; privacy label and Data safety form matching reality; Google Play health apps declaration; "not a medical device" disclaimer.

- **Data rules per country:** see the MENA research tab of the proposal. In-country hosting (self-hosted Supabase) before any readable health data goes to the cloud; PDPC licence in Egypt before AI Doctor or cloud OCR process health data abroad.
- **Audit:** local access log of shares and AI runs; penetration test before cloud features launch.

Testing, CI/CD and environments

Area	How
Unit tests	Domain classes: schedules, rule engine, OCR drug matcher, growth z-scores, credit logic
Widget / golden tests	Key screens in Arabic and English, normal and large text, light and dark
Integration tests	Scan → medicine → reminder; backup → restore on a clean device
Device lab	3+ budget Android phones (Xiaomi, Oppo, Samsung) for alarm reliability; one older iPhone
Edge Functions	Deno tests + a local Supabase stack (`supabase start`) in CI
AI Doctor	Fixed set of test snapshots with expected findings; checks that the output validates and stays cautious
CI	GitHub Actions: lint, tests, build on every pull request
Builds and release	Codemagic for iOS + Android, Fastlane to TestFlight and Play internal testing
Environments	`dev` (local Supabase), `staging` and `prod` Supabase projects; feature flags per environment

Apple App Store (iOS)

The same Flutter code ships to the App Store; what differs is account setup, a few iOS-only limits, and stricter review for medical apps.

Account and submission

- **Publish as a company, not as an individual.** Apple says healthcare apps "should be submitted by a legal entity" (guideline 5.1.1(ix)). Enroll Opsedia in the Apple Developer Program as an organization ($99/year; needs a D-U-N-S number).
- iOS builds need a Mac, or Codemagic in the cloud. Beta testing goes through TestFlight.
- Give App Review a demo account with sample family data, plus notes explaining offline storage, OCR and the AI Doctor.

Review rules that affect Usra

Rule	What it means for Usra
1.4.1 Medical apps get extra scrutiny	Remind users to check with a doctor; no accuracy claims we can't support; growth checks cite WHO standards
1.4.2 Drug dosage calculators must come from a manufacturer, hospital, pharmacy or other approved entity, or have regulatory approval	**Keep the dose calculator off on iOS** unless done with an approved partner
5.1.1(v) Account deletion	In-app "Delete account and all data" (already designed)
5.1.3 Health data	No health data in iCloud; never used for ads; AI Doctor only with permission, for health management
In-app purchase	AI credits and Usra Plus must use Apple's in-app purchase (via RevenueCat)
Privacy	Accurate privacy "nutrition" label and privacy manifest for every SDK; no tracking, so no tracking prompt needed

iOS-specific engineering

- **Alarms:** iOS doesn't allow Android-style full-screen alarms from normal notifications. On iOS 26 and later, use **AlarmKit**, Apple's framework that gives apps system-level alarms (Apple docs; reported to ring through Silent and Focus, to be confirmed in testing). On older iOS, use Time Sensitive notifications, plus the 64-pending-notification limit handled by the scheduler.
- **Widget:** the "Today's medicines" widget is written in Swift with WidgetKit; the "Taken" button uses App Intents.
- **Apple Health:** importing vitals uses HealthKit, which needs its own permission screen and a clear reason in the App Store listing.

- **OCR:** on iOS, Apple Vision can read more languages than ML Kit; test whether it reads Arabic prescriptions well enough to reduce cloud fallback.

Cost and payouts: Apple's commission on AI credits and subscriptions is 15% for small developers in the App Store Small Business Program (30% otherwise). Payouts need a bank account for the company.

Build plan

Milestone	Scope	Exit criteria
M0 Foundations	Repo, Flutter app shell, module system, design system (Nile blue, severity colours, Dr. Usra), Drift + SQLCipher, Arabic/English, CI	App runs on Android + iOS; tests green
M1 Core records	Family profiles, timeline, record types, doctors, insurance cards + WhatsApp share, attachments	Can record a full visit with photos
M2 Medicines + reminders	OCR pipeline (on-device), medicines, schedules, alarms, widget, dose log	Reminders fire reliably on 3 budget Android phones
M3 Family care	Child growth + vaccines, infant log, pregnancy, elder Simple Mode, emergency card, follow-up rules	Validation testers complete the elderly tasks
M4 Money + backup	Spending dashboard, encrypted backup/restore, recovery key	Restore works on a new phone
M5 Launch v1	Store listings, privacy forms, legal review, beta in Egypt	Live on both stores
M6 Cloud + AI	Supabase accounts, E2EE sync, caregiver alerts, cloud OCR fallback, AI Doctor + credits/subscription	Legal gates cleared; AI safety tests pass
M7 Partners	Clinic portal, appointment reminders, self-hosted Supabase in-region	First pilot clinic live

Durations depend on team size; the feasibility review estimated ~3–5 months full-time for one developer to reach M5.

Risks and open questions

Risk	Mitigation
No Supabase Cloud region in the Middle East	Keep health data on phone/E2EE; self-host Supabase in-region for readable data later
ML Kit can't read Arabic	Cloud fallback with consent; most drug names on boxes are Latin
Alarms killed on budget Android	Exact alarms, full-screen intents, per-brand guidance, device lab testing
AI Doctor counted as a medical device or blocked by data-transfer rules	Rules-first design, de-identified snapshots, launch per country only after legal review
LLM cost per run	Token budget, caching, credit pricing above cost
Store review of a health app with AI	Clear disclaimers, consent screens, accurate privacy forms, test accounts for reviewers

Open questions

- [ ] Flutter confirmed, or does the founding developer prefer React Native?
- [ ] Which LLM provider and region for AI Doctor (and is an in-region endpoint available)?
- [ ] AI credit prices and Usra Plus price per market
- [ ] Which cloud and region for self-hosted Supabase later (Egypt / UAE / KSA)?
- [ ] Source and licence of the Egyptian drug list